Branch-and-cut

Outline

- Why branch-and-cut algorithms?
- Branch-and-cut algorithms
- Global and local valid inequalities
- Cut-and-branch and branch-and-cut
  - The cut pool
  - High-level pseudocode
- Worked example: cut-and-branch with Gomory cuts
- Local cuts: an example
- Practical aspects
  - Cut selection
  - Which cuts
  - Upper bound and gap
- Typical mistakes
- In summary

Learning objectives

At the end of this chapter you will be able to:

- explain how cutting planes and branching operations complement each other;
- distinguish global and local valid inequalities, and say where a cut can be used;
- describe cut-and-branch and branch-and-cut algorithms, and the role of the cut pool;
- execute a cut-and-branch algorithm with Gomory cuts by hand on a small ILP problem and compare the size of the tree with and without cuts;
- generate a local cut at a branching node and re-optimize with the dual simplex algorithm.

Prerequisites

From this module:

- The chapters «Valid inequalities» and «Cover inequalities»: valid inequalities, and the separation of a violated inequality of a family
- The chapter «Cutting plane algorithm»: Gomory fractional cuts, and the re-optimization of the LP problem with the dual simplex algorithm after the insertion of a cut
- The chapter «LP-based branch-and-bound»: the list of LP problems, the pruning criteria, and the branching-variable and branching-node selection strategies

Why branch-and-cut algorithms?

Introduction: Why branch-and-cut algorithms?

- We have seen two exact methods for ILP problems: the cutting plane algorithm and the branch-and-bound algorithm
- Each of them has a weak point, and the weak points are different
- We see why it pays to combine them, and we meet the challenge of the chapter

Two exact methods

- We consider ILP problems in **canonical maximization form with integer input data**, as in the chapters on cutting planes and on branch-and-bound
- The two exact methods seen so far attack the ILP problem from two different sides:
  - a **cutting plane algorithm** adds valid inequalities and moves the LP relaxation towards the convex hull of the integer solutions; the upper bound decreases
  - a **branch-and-bound algorithm** divides the feasible region into smaller LP problems and enumerates them implicitly; the search space is partitioned

Two weak points

- Both methods have a weak point
- Gomory fractional cuts lose effectiveness after the first iterations (**tailing off**)
- A branch-and-bound algorithm started from a weak LP relaxation needs a large tree

A challenge

Consider the problem $P10$ of the catalogue of MIP problems:
its branch-and-bound tree has 5 nodes
with the rules of the worked example.

How many nodes are left
if we add **one Gomory cut** before branching?

- The worked example of this chapter answers the question

Cuts and branching together

- A **branch-and-cut algorithm** is a branch-and-bound algorithm in which the LP problems of the branching nodes are strengthened with cutting planes before branching
- Cuts decrease the bounds of the nodes, so more nodes are pruned by bound
- Branching guarantees termination when the cuts are no longer effective
- All modern MIP solvers are based on branch-and-cut algorithms (see the chapter «What a MIP solver does»)

Summary

- Cutting planes **decrease the upper bound**, but they suffer from tailing off
- Branch-and-bound **partitions the search space**, but with a weak LP relaxation the tree is large
- A **branch-and-cut algorithm** uses both: cuts at the nodes, branching to terminate
- The challenge: how much does one Gomory cut reduce the tree of $P10$?

Branch-and-cut algorithms

Introduction: Branch-and-cut algorithms

- We give the formal definition of a branch-and-cut algorithm
- We see how cutting planes and branching operations complement each other
- This is the scheme on which every modern MIP solver is built

Definition 1: branch-and-cut algorithm

Given an ILP problem in canonical maximization form with integer input data, a ***branch-and-cut algorithm*** *is a branch-and-bound algorithm in which the LP problems of the branching nodes are strengthened with cutting planes before branching.*

Two methods that complement each other

- A **cutting plane algorithm** moves the LP relaxation towards the convex hull of the integer solutions, but Gomory fractional cuts lose effectiveness after the first iterations (**tailing off**)
- A **branch-and-bound algorithm** partitions the feasible region into smaller LP problems, but started from a weak LP relaxation it needs a large tree

Why the combination works

Cuts decrease the bounds of the nodes,
so more nodes are **pruned by bound**;

branching guarantees **termination**
when the cuts are no longer effective.

- All modern MIP solvers are based on branch-and-cut algorithms (see the chapter «What a MIP solver does»)

Summary

- A branch-and-cut algorithm is a **branch-and-bound algorithm** whose node LP problems are **strengthened with cuts** before branching
- Cuts lower the bounds of the nodes; branching guarantees termination
- It is the scheme of all modern MIP solvers

Global and local valid inequalities

Introduction: Global and local valid inequalities

- At a node of the tree, the LP problem contains branching constraints
- A cut derived with these constraints may be valid only below that node
- We distinguish **global** and **local** cuts, and see where each of them can be used: a mistake here can remove the optimal solution

The integer solutions of a node

- In a branch-and-bound algorithm, the LP_k problem of a branching node k is the LP relaxation with the branching constraints of the path from the root to node k
- We denote by F the set of integer feasible solutions of the ILP problem
- We denote by $F_k \subseteq F$ the set of those that satisfy the branching constraints of node k
- At the root node $F_1 = F$

Definition 2: global and local valid inequalities

A linear inequality $\alpha' \boldsymbol{x} \leq \beta$ is called:

1. *a **globally valid inequality** (or **global cut**) if it is satisfied by all the solutions in F;*
2. *a **locally valid inequality** for the branching node k (or **local cut**) if it is satisfied by all the solutions in F_k.*

Global cuts and local cuts

- Since $F_k \subseteq F$, every global cut is also a local cut of every node
- The converse is false: a local cut can remove integer feasible solutions that belong to other branches of the tree
- A local cut of node k is valid for all the nodes of the subtree rooted at k, and it must not be used outside this subtree

When is a cut global?

A cut derived only from the original constraints
(and from other global cuts) is **global**.

A cut whose derivation uses a branching constraint
is, in general, only **local**.

Gomory cuts and cover inequalities

- A Gomory fractional cut generated at the root node is **global**: it is derived from the original constraints alone
- The same holds for an inequality of a family (such as the cover inequalities of a constraint with positive coefficients) separated from them
- A Gomory fractional cut generated by a row that contains the slack variable of a branching constraint with a non-zero coefficient is treated as **local**

Summary

Global and local cuts:

- Global: satisfied by all the solutions in F; local for node k: by all the solutions in F_k
- Every global cut is local for every node; the converse is false

Where a cut can be used:

- A local cut of node k is used only in the subtree rooted at k
- A cut whose derivation uses a branching constraint is, in general, only local

Cut-and-branch and branch-and-cut

Introduction: Cut-and-branch and branch-and-cut

- Cuts can be generated only at the root, or also at the other nodes of the tree
- We define the two algorithms, the **cut pool** that stores the cuts, and the sub-procedure that solves a node
- The pseudocode is that of branch-and-bound: only the solution of a node changes

Definition 3: cut-and-branch algorithm

*A **cut-and-branch algorithm** executes a cutting plane algorithm at the root node for a limited number of **rounds of cuts**, adds the cuts found to the ILP problem, and then executes a branch-and-bound algorithm on the strengthened problem. No cuts are generated at the other nodes.*

Cuts at the root and at the other nodes

- All the root cuts are global: at the root node no branching constraint exists
- In a **branch-and-cut algorithm** cuts are generated also at the other nodes (at all of them, or at some of them, for example every ten levels of the tree)
- The cuts can then be global or local

When to stop cutting at a node

At each node, the generation of cuts stops when:

- the solution of the LP_k problem is integer or the node can be pruned;
- no violated cut is found;
- a maximum number of rounds is reached;
- the decrease of $z(\mathrm{LP}_k)$ in the last rounds is small (tailing off).

The cut pool (1/2)

- Cuts are stored in a data structure called the **cut pool**
- Before calling the (more expensive) cut-generation procedures, the algorithm checks whether a cut of the pool is violated by $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$
- Global cuts are stored in a **global pool**, usable at every node
- Local cuts are stored with their node and are inherited only by its descendants

The cut pool (2/2)

- Each cut adds a constraint to the LP problems
- Cuts that stay inactive for many iterations are removed from the LP problem and kept only in the pool

High-level pseudocode

- The branch-and-cut algorithm has the pseudocode of the branch-and-bound algorithm:
  - same branching operations
  - same pruning criteria
  - same list L
- Only the **Branching Node Solution Sub-Procedure** changes: it is shown on the next two slides

Branching Node Solution Sub-Procedure (1/2)

Input: An LP_k problem associated with a branching node, the cut pool, and a maximum number R of rounds of cuts.

Step 1: **Bounding operations**

Solve the LP_k problem.

(a) if the LP_k problem is infeasible, then `STOP`.

(b) if $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ is integer, then, if $z(\mathrm{LP}_k) > \mathrm{LB}$, update the **lower bound** and the **incumbent solution**, remove from the list L all LP problems with $\lfloor z(\mathrm{LP}) \rfloor \leq \mathrm{LB}$ and `STOP` (also when $z(\mathrm{LP}_k) \leq \mathrm{LB}$).

(c) if $\lfloor z(\mathrm{LP}_k) \rfloor \leq \mathrm{LB}$, then `STOP`.

Branching Node Solution Sub-Procedure (2/2)

Step 2: **Cut generation**

If R rounds of cuts have been executed at node k, go to Step 3. Otherwise:

(a) look in the cut pool for cuts violated by $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$; if none is found, generate new violated cuts and store them in the pool (global cuts in the global pool, local cuts with node k);

(b) if no violated cut exists, go to Step 3. Otherwise add the cuts to the LP_k problem, re-optimize it and go to Step 1.

Step 3: **Insertion into the list**

Insert the LP_k problem (with its cuts) into the list L and `STOP`.

Special cases and re-optimization

- With $R = 0$ at every node except the root we obtain a **cut-and-branch algorithm**
- With $R = 0$ at every node we obtain the **branch-and-bound algorithm**
- The pseudocode does not say how the cuts are generated or how the LP problem is re-optimized
- After the insertion of a cut the dictionary is **dual feasible**, so the **dual simplex algorithm** starts from the optimal dictionary of the previous problem, as after a branching constraint

Summary

Two algorithms:

- Cut-and-branch: rounds of cuts only at the root, then branch-and-bound; all cuts are global
- Branch-and-cut: cuts also at the other nodes, global or local

The cut pool:

- Checked before generating new cuts; global pool and local cuts stored with their node

The sub-procedure:

- Bounding, cut generation for at most R rounds, insertion into L; re-optimization with the dual simplex algorithm

Worked example: cut-and-branch with Gomory cuts

Introduction: Worked example

- We run the branch-and-bound algorithm on the problem $P10$, first without cuts and then after one Gomory cut at the root
- We compare the sizes of the two trees, and see what a second round of cuts does
- A second problem, $P2$, shows that a cut does not always reduce the tree

The problem $P10$

- We use the problem $P10$ of the catalogue of MIP problems:

$$\begin{array}{rrcrcr} \max & 5\,x_1 & + & 2\,x_2 & & \\ \text{subject to} & -x_1 & + & 3\,x_2 & \leq & 12 \\ & x_1 & + & 2\,x_2 & \leq & 12 \\ & 3\,x_1 & - & 2\,x_2 & \leq & 8 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

- Its unique optimal solution is $\tilde{\boldsymbol{x}}(\mathrm{ILP}) = (4, 4)$, with $z(\mathrm{ILP}) = 28$

The rules of the executions

- The cuts are the Gomory fractional cuts of the chapter on the cutting plane algorithm, with the same rule: the row with the largest fractional right-hand side, and the smallest index in case of a tie
- Since the data are integer, the bound of a node is $\lfloor z(\mathrm{LP}_k) \rfloor$
- All the executions use the same rules:
  - the smallest-index rule as branching-variable selection strategy
  - the LIFO rule as branching-node selection strategy
  - the down-branching node solved first
  - $\mathrm{LB} = -\infty$ at the start

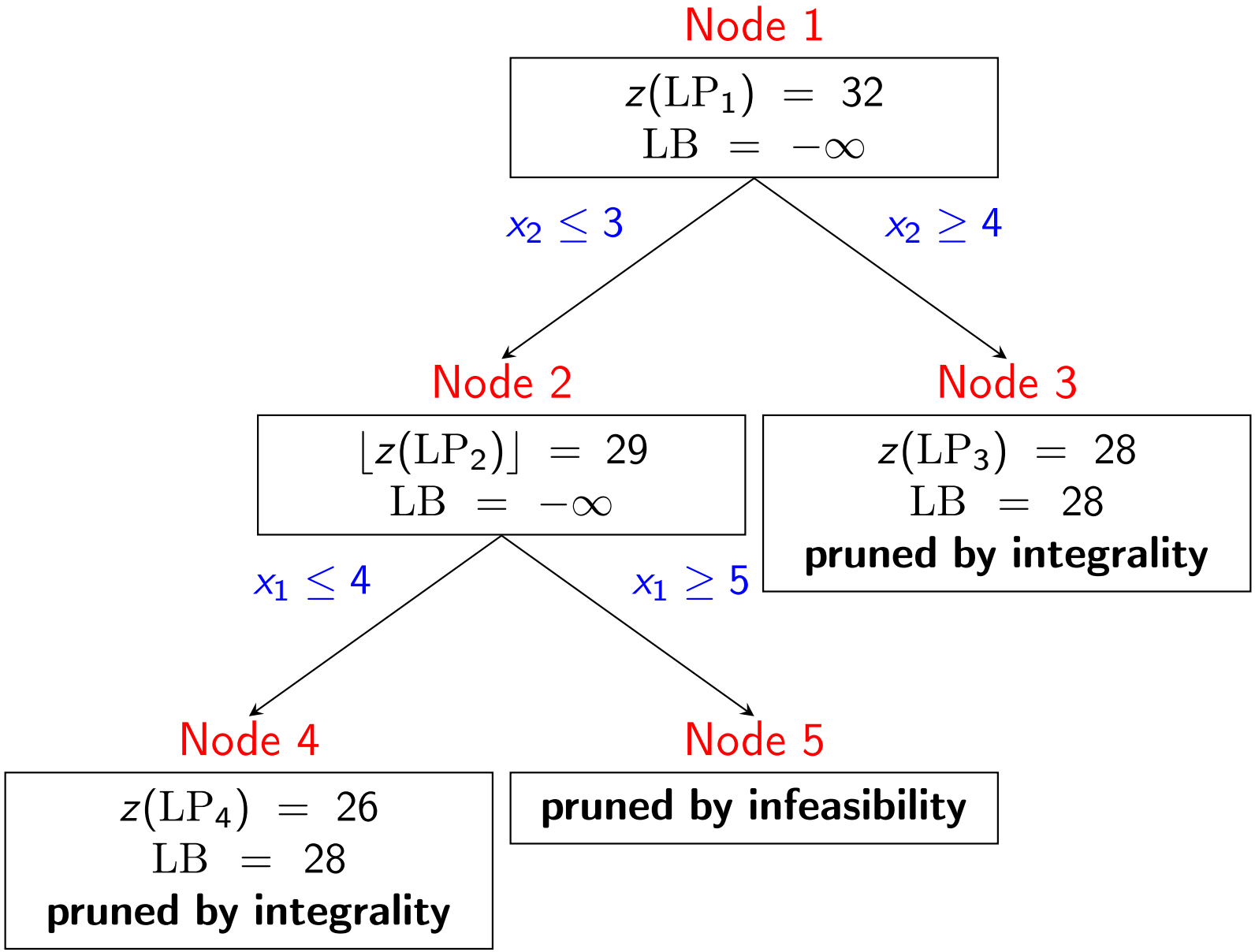
Example: branch-and-bound without cuts (1/3)

- The LP_1 problem is the LP relaxation of $P10$
- An optimal solution is

$$\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (5, \tfrac{7}{2}), \qquad z(\mathrm{LP}_1) = 32, \qquad \mathrm{UB} = 32$$

- The branch-and-bound algorithm solves 5 LP problems: the tree and the table of the execution are on the next two slides

Example: branch-and-bound without cuts (2/3)



Example: branch-and-bound without cuts (3/3)

Node	Parent	Branching	$\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	$z(\mathrm{LP}_k)$	LB	Outcome
1	–	–	$(5, \frac{7}{2})$	32	$-\infty$	branching on x_2
2	1	$x_2 \leq 3$	$(\frac{14}{3}, 3)$	$\frac{88}{3}$	$-\infty$	branching on x_1
3	1	$x_2 \geq 4$	$(4, 4)$	28	28	pruned by integrality
4	2	$x_1 \leq 4$	$(4, 3)$	26	28	pruned by integrality
5	2	$x_1 \geq 5$	–	–	28	pruned by infeasibility

Now we add one Gomory cut at the root
before branching, and we run
the same branch-and-bound algorithm again.

Example: cut-and-branch with one Gomory cut (1/5)

- **Root: cut generation.** The two rows of the optimal dictionary of LP_1 with a fractional right-hand side, the rows of x_2 and of x_3, have the same right-hand side $\frac{1}{2}$
- We use the row of x_2, whose index is smaller
- With the slack variables $x_4 = 12 - x_1 - 2x_2$ and $x_5 = 8 - 3x_1 + 2x_2$ the cut is

$$\frac{3}{8}\,x_4 + \frac{7}{8}\,x_5 \geq \frac{1}{2} \qquad \Longleftrightarrow \qquad 3\,x_1 - x_2 \leq 11.$$

- It is a global cut, and it cuts off $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$: $3 \cdot 5 - \frac{7}{2} = \frac{23}{2} > 11$

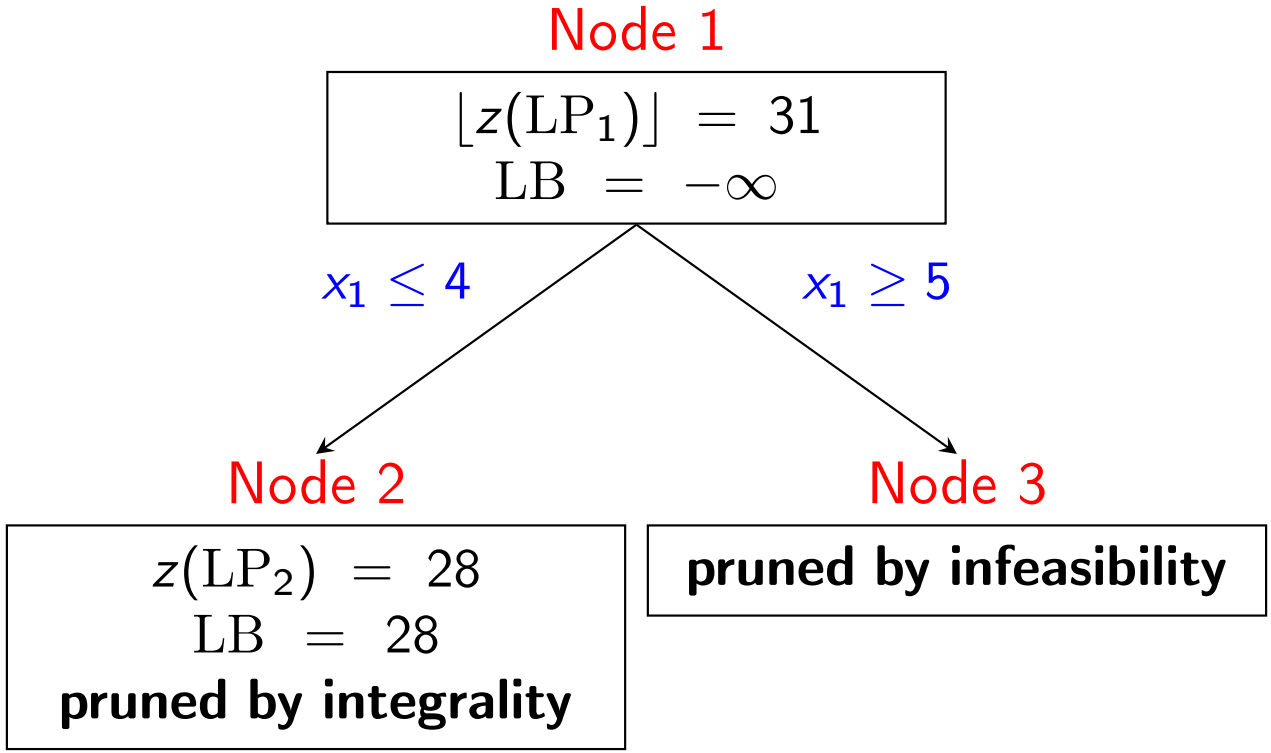
Example: cut-and-branch with one Gomory cut (2/5)

- **Root: re-optimization.** We add the cut to the LP_1 problem and we re-optimize it with the dual simplex algorithm
- An optimal solution is

$$\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (\tfrac{34}{7}, \tfrac{25}{7}), \qquad z(\mathrm{LP}_1) = \tfrac{220}{7}, \qquad \mathrm{UB} = 31.$$

- **Branch-and-bound.** The branch-and-bound algorithm now starts from this LP_1 problem
- Every node inherits the cut. It solves 3 LP problems instead of 5: tree and table on the next slides

Example: cut-and-branch with one Gomory cut (3/5)



Example: cut-and-branch with one Gomory cut (4/5)

Node	Parent	Branching	$\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	$z(\mathrm{LP}_k)$	LB	Outcome
1	–	–	$(\frac{34}{7}, \frac{25}{7})$	$\frac{220}{7}$	$-\infty$	branching on x_1
2	1	$x_1 \leq 4$	$(4, 4)$	28	28	pruned by integrality
3	1	$x_1 \geq 5$	–	–	28	pruned by infeasibility

Example: cut-and-branch with one Gomory cut (5/5)

- One Gomory cut reduces the number of LP problems from 5 to 3
- The cut does not find the optimal solution: it moves the optimal solution of the root LP problem
- So the algorithm branches on x_1, and the node $x_1 \leq 4$ gives the optimal solution at once

A second round of cuts

- A second round of cuts, in the same cut-and-branch algorithm, gives an integer solution at the root
- The row of x_1 has the largest right-hand side $\frac{6}{7}$
- With the slack variable $x_6 = 11 - 3x_1 + x_2$ of the first cut the new cut is

$$\frac{1}{7}\, x_4 + \frac{2}{7}\, x_6 \geq \frac{6}{7} \qquad \Longleftrightarrow \qquad x_1 \leq 4.$$

The root becomes integer

- The optimal solution of the new LP_1 problem is $(4, 4)$, with $z(\mathrm{LP}_1) = 28$
- The node is pruned by integrality, and the algorithm stops with $\mathrm{LB} = \mathrm{UB} = 28$ after solving one LP problem
- The table on the next slide compares the three executions on $P10$

Three executions on $P10$

Cuts added at the root	$z(\mathrm{LP}_1)$	UB at the root	LP problems solved
none (branch-and-bound)	32	32	5
$3x_1 - x_2 \leq 11$	$\frac{220}{7}$	31	3
$3x_1 - x_2 \leq 11$ and $x_1 \leq 4$	28	28	1

Stronger cuts, smaller trees?

- On $P10$: the stronger the cuts, the smaller the tree, until it reduces to the root node
- This is not the rule
- We run the same algorithm on the problem $P2$ of the catalogue, for an increasing number of rounds of Gomory cuts

The problem $P2$

- The problem $P2$ of the catalogue of MIP problems:

$$\begin{array}{rrcrcr} \max & x_1 & + & 3\,x_2 & & \\ \text{subject to} & -4\,x_1 & + & 10\,x_2 & \leq & 35 \\ & 3\,x_1 & + & x_2 & \leq & 12 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

- Its optimal value is $z(\mathrm{ILP}) = 14$

Rounds of Gomory cuts on $P2$

Rounds	Last cut added	$z(\mathrm{LP}_1)$	UB at the root	LP problems solved
0	–	16	16	4
1	$-3\,x_1 + 10\,x_2 \leq 37$	$\frac{524}{33}$	15	4
2	$x_1 + 3\,x_2 \leq 15$	15	15	4
3	$-x_1 + 5\,x_2 \leq 19$	15	15	4
4	$x_2 \leq 4$	$\frac{44}{3}$	14	2
5	$x_1 + x_2 \leq 6$	14	14	1

Tailing off on $P2$

- The first three cuts decrease $z(\mathrm{LP}_1)$ from 16 to 15 but the tree has always 4 nodes
- 5 rounds are needed to reach the root node
- The upper bound improves slowly, as in the **tailing off** of the cutting plane algorithm
- A cut-and-branch algorithm must decide when to stop cutting and start branching: the cost of each round is paid at every node of the tree

Summary

On $P10$:

- Branch-and-bound without cuts: 5 LP problems
- One Gomory cut at the root, $3x_1 - x_2 \leq 11$: 3 LP problems
- A second cut, $x_1 \leq 4$: the root is integer, 1 LP problem

On $P2$:

- The first three cuts leave the tree at 4 nodes; 5 rounds reach the root node
- When to stop cutting and start branching is a decision of the algorithm

Local cuts: an example

Introduction: Local cuts: an example

- We go back to the car example, at a node below the root
- We derive a local cut from a branching constraint, add it and re-optimize with the dual simplex algorithm
- Then we see a Gomory cut at the same node that the algorithm must treat as local

The car example

- We use the car example (x_1 sports cars and x_2 family cars):

$$\begin{array}{rrcrcr} \max & 3\,x_1 & + & 2\,x_2 & & \\ \text{subject to} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\ & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

- In the branch-and-bound algorithm without cuts, node 3 is the up-branching child of the root, with the branching constraint $x_1 \geq 4$ and the slack variable $x_5 = x_1 - 4$

The optimal dictionary of LP_3

- The optimal dictionary of the LP_3 problem, with $\tilde{\boldsymbol{x}}(\mathrm{LP}_3) = (4, \frac{5}{4})$ and $z(\mathrm{LP}_3) = \frac{29}{2}$, is:

$$
\begin{cases}
x_1 = 4 + x_5 \\
x_2 = \frac{5}{4} - \frac{1}{4} x_3 - \frac{5}{2} x_5 \\
x_4 = \frac{31}{2} + \frac{7}{2} x_3 + 31 x_5 \\
z = \frac{29}{2} - \frac{1}{2} x_3 - 2 x_5
\end{cases}
$$

Example: a local cut at node 3 (1/7)

- **A simple local cut.** From $10\,x_1 + 4\,x_2 \leq 45$ and the branching constraint $x_1 \geq 4$ we have

$$4\,x_2 \leq 45 - 10\,x_1 \leq 45 - 10 \cdot 4 = 5 \quad \Longrightarrow \quad x_2 \leq \tfrac{5}{4} \quad \Longrightarrow \quad x_2 \leq 1.$$

Example: a local cut at node 3 (2/7)

- The inequality $x_2 \leq 1$ is valid for all the solutions in F_3, but it is **not globally valid**
- The integer feasible solution $(3, 2)$, of value 13, violates it
- It is a local cut of node 3
- We add it to the LP_3 problem, with the slack variable $x_6 = 1 - x_2$, and we re-optimize

Example: a local cut at node 3 (3/7)

- The cut is inserted with the slack variable $x_6 \geq 0$. The new row is

$$x_6 = -\frac{1}{4} + \frac{1}{4}\, x_3 + \frac{5}{2}\, x_5.$$

- The dictionary on the next slide is **dual feasible** but not primal feasible, since

$$\bar{b}_6 = -\frac{1}{4} < 0$$

Example: a local cut at node 3 (4/7)

$$
\begin{cases}
x_1 = 4 + x_5 \\
x_2 = \frac{5}{4} - \frac{1}{4} x_3 - \frac{5}{2} x_5 \\
x_4 = \frac{31}{2} + \frac{7}{2} x_3 + 31 x_5 \\
x_6 = -\frac{1}{4} + \frac{1}{4} x_3 + \frac{5}{2} x_5 \\
z = \frac{29}{2} - \frac{1}{2} x_3 - 2 x_5
\end{cases}
$$

Example: a local cut at node 3 (5/7)

- **Dual pivoting operation.** The pivot row is x_6, the only basic variable with a negative value
- The ratios of the non-basic variables with $\bar{a}_{6j} < 0$ are

$$\frac{\bar{c}_3}{\bar{a}_{6,3}} = \frac{-\frac{1}{2}}{-\frac{1}{4}} = 2, \qquad \frac{\bar{c}_5}{\bar{a}_{6,5}} = \frac{-2}{-\frac{5}{2}} = \frac{4}{5}.$$

- The pivot column is x_5, with $s_m = \frac{4}{5}$, and the objective value becomes

$$\bar{z} + s_m \bar{b}_6 = \frac{29}{2} + \frac{4}{5} \cdot \left(-\frac{1}{4}\right) = \frac{143}{10}$$

Example: a local cut at node 3 (6/7)

- The new dictionary is primal and dual feasible, hence optimal:

$$
\begin{cases}
x_1 = \frac{41}{10} - \frac{1}{10} x_3 + \frac{2}{5} x_6 \\
x_2 = 1 - x_6 \\
x_4 = \frac{93}{5} + \frac{2}{5} x_3 + \frac{62}{5} x_6 \\
x_5 = \frac{1}{10} - \frac{1}{10} x_3 + \frac{2}{5} x_6 \\
z = \frac{143}{10} - \frac{3}{10} x_3 - \frac{4}{5} x_6
\end{cases}
$$

Example: a local cut at node 3 (7/7)

- The solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_3) = (\frac{41}{10}, 1)$, with value $\frac{143}{10}$, is the same that the down-branching child of node 3 gives
- Here the local cut works as a branching constraint, without creating a new node

The cut $x_2 \leq 1$ was derived by hand.

What about a Gomory fractional cut generated from the dictionary of node 3?

Example: a Gomory fractional cut at node 3 (1/2)

- The row of x_2 of the dictionary of node 3 gives the Gomory fractional cut

$$\frac{1}{4}x_3 + \frac{1}{2}x_5 \geq \frac{1}{4}$$

- It contains the slack variable x_5 of the branching constraint $x_1 \geq 4$, so its derivation uses this constraint and the algorithm treats it as a **local cut**

Example: a Gomory fractional cut at node 3 (2/2)

- In the space of the original variables ($x_3 = 45 - 10\,x_1 - 4\,x_2$, $x_5 = x_1 - 4$, multiplied by 4) it reads

$$2\,x_1 + x_2 \leq 9$$

- Enumerating the 16 integer feasible solutions we see that this inequality is valid for all of them

- The algorithm cannot make this check at a node, so it keeps the cut local

Summary

A local cut at node 3:

- From $10x_1 + 4x_2 \leq 45$ and $x_1 \geq 4$: the cut $x_2 \leq 1$, valid in F_3 but violated by $(3, 2)$
- One dual pivoting operation gives $(\frac{41}{10}, 1)$, with value $\frac{143}{10}$: the cut works as a branching constraint

A Gomory cut at node 3:

- It contains the slack variable x_5 of $x_1 \geq 4$: it is treated as local
- It is in fact global ($2x_1 + x_2 \leq 9$), but the algorithm cannot know it

Practical aspects

Introduction: Practical aspects

- A real branch-and-cut algorithm must choose which cuts to add, among many
- We see how cuts are measured (**efficacy**), which families of cuts are used, and how the cuts change the upper bound and the gap
- These choices decide the performance of a MIP solver

Cut selection

- At each round many violated cuts can be available
- Adding all of them makes the LP problems large and slow to re-optimize
- A branch-and-cut algorithm selects a subset

The efficacy of a cut

- The **efficacy** of a cut $\boldsymbol{\alpha}'\boldsymbol{x} \leq \beta$ at the fractional solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ is the Euclidean distance of $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ from the hyperplane $\boldsymbol{\alpha}'\boldsymbol{x} = \beta$:

$$\text{efficacy} = \frac{\boldsymbol{\alpha}'\,\tilde{\boldsymbol{x}}(\mathrm{LP}_k) - \beta}{\|\boldsymbol{\alpha}\|_2}.$$

- Unlike the violation $\boldsymbol{\alpha}'\tilde{\boldsymbol{x}}(\mathrm{LP}_k) - \beta$, the efficacy does not change if the cut is multiplied by a positive constant

- Selection rules combine efficacy with a measure of the **parallelism** of the cuts

Efficacy and violation on $P10$

- At $\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (5, \frac{7}{2})$ of $P10$:
- the cut $3x_1 - x_2 \leq 11$ has violation $\frac{1}{2}$ and efficacy

$$\frac{1}{2\sqrt{10}} \approx 0.158$$

- the same cut multiplied by 2, $6x_1 - 2x_2 \leq 22$, has violation 1 and the same efficacy

$$\frac{1}{\sqrt{40}} \approx 0.158$$

Which cuts

- Any family of valid inequalities with a separation procedure can be used:
  - Gomory fractional cuts (for ILP problems with integer data)
  - cover inequalities for the constraints $\sum_j a_j x_j \leq b$ with positive coefficients of a MIP problem with binary variables
  - families that exploit the structure of a given model

- Gomory fractional cuts come from the rows of the optimal dictionary, so they are always available

- In floating-point arithmetic their coefficients are affected by rounding errors, and solvers discard the cuts with very large or very small coefficients

Upper bound and gap

- As in the branch-and-bound algorithm, if the list L is not empty we have

$$z(\mathrm{ILP}) \quad \leq \quad \mathrm{UB} \quad = \quad \max_{k \in L} \{ \, \mathrm{UB}(\mathrm{LP}_k) \, \},$$

  where $z(\mathrm{LP}_k)$ is now the optimal value of the LP_k problem after the cuts

- Cuts decrease the values $z(\mathrm{LP}_k)$ and, accordingly, the upper bound UB and the optimality gap

$$\frac{\mathrm{UB} - \mathrm{LB}}{|\mathrm{LB}|}$$

Summary

Cut selection:

- Only a subset of the violated cuts is added
- Efficacy: distance of $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ from the hyperplane of the cut, invariant under scaling; combined with parallelism

Which cuts:

- Gomory fractional cuts, cover inequalities, families of a given model; numerically unsafe cuts are discarded

Bounds:

- Cuts decrease $z(\mathrm{LP}_k)$, the upper bound UB and the gap

Typical mistakes

Introduction: Typical mistakes

- We list the mistakes that are made most often with branch-and-cut algorithms
- They concern the validity of the cuts, how many cuts to add, how to compare them, and what cuts change
- Knowing them helps to read and solve the exercises correctly

Typical mistakes

Observation 1: Typical mistakes (1/3)

- ***Using a local cut outside its subtree.*** *A cut derived with a branching constraint of node k is valid only in the subtree of node k: in another branch it can remove an optimal solution.*
- ***Treating a cut as global without a proof.*** *A Gomory fractional cut generated at a node can be global by chance (the Gomory cut at node 3 of the car example); the algorithm keeps it local unless its derivation does not use branching constraints.*

Typical mistakes

Observation: Typical mistakes (2/3)

- ***Adding every violated cut.*** *Too many cuts make the LP problems slow; a few effective cuts per round are better, and inactive cuts are removed.*
- ***Believing that every cut reduces the tree.*** *On P2 the first three Gomory cuts leave the tree at 4 nodes (table of the worked example); the quality of the cuts matters.*

Typical mistakes

Observation: Typical mistakes (3/3)

- ***Choosing the cut with the largest efficacy as the strongest.*** *Efficacy is measured at the current point only: a dominated cut can have a larger efficacy at that point.*
- ***Forgetting that cuts decrease the bound, not the optimal value.*** *The optimal value* $z(\mathrm{ILP})$ *never changes: only* $z(\mathrm{LP}_k)$ *and the upper bound do.*

Summary

Typical mistakes:

- A local cut is used only in its subtree, and a cut is global only with a proof
- Few effective cuts are better than every violated cut
- Not every cut reduces the tree, and efficacy is measured only at the current point
- Cuts change the bound, never $z(\mathrm{ILP})$

In summary

Introduction: In summary

- We collect the key ideas of the chapter
- We close with the dashboard of $P10$: the bounds found in this chapter

Summary (1/3)

- A branch-and-cut algorithm strengthens the LP problem of every node with cuts before branching
- A global cut is valid for all the integer solutions; a local cut only for those of a subtree, and it is stored with its node
- A cut-and-branch algorithm generates cuts only at the root; the cuts are then global

Summary (2/3)

- After a cut, the LP problem is re-optimized with the dual simplex algorithm from the previous dictionary
- Cuts reduce the size of the tree, but only effective cuts do: on $P10$, one Gomory cut reduces the tree from 5 to 3 LP problems, and two cuts to 1
- How MIP solvers combine these ideas is the topic of the chapter «MIP solvers»

Summary (3/3)

Dashboard of $P10$. The lower and upper bounds on $z(\mathrm{ILP}) = 28$ found in this chapter:

Stage	Lower bound	Upper bound	From
LP relaxation of $P10$	—	32	$z(\mathrm{LP}) = 32$, $\tilde{\boldsymbol{x}}(\mathrm{LP}) = (5, \frac{7}{2})$
Branch-and-bound	28	28	5 LP problems solved
After the first Gomory cut	—	31	$z(\mathrm{LP}_1) = \frac{220}{7}$
Cut-and-branch with one cut	28	28	3 LP problems solved
After the second Gomory cut	28	28	$\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (4, 4)$ is integer, 1 LP problem solved